Stop Forcing `std::move` to Pessimize

Document #:	P2991R0
Date:	2023-10-11
Project:	Programming Language C++
Audience:	SG20 (Education)

Reply-to:	Brian Bi <bbi10@bloomberg.net>

Contents

1 Abstract

2 The problem

3 The solution

4 Should the core language have special cases for library entities?

5 How many programs will be broken by this change?

6 Alternative solutions

7 Wording

8 References

§ 1 Abstract

Those who learn C++ are taught to use `std::move` when initializing a new object that needs to hold the value of a given source lvalue and the source object no longer needs to hold that value. However, an exception to this guideline is soon introduced: when returning the name of a local variable, applying `std::move` can inhibit the named-return-value optimization (NRVO). This paper proposes to eliminate this exception by making the expression `std::move(x)` *NRVO eligible* when `x` is NRVO eligible. Doing so will improve the teachability of move semantics in C++.

§ 2 The problem

The following code illustrates a familiar pattern in which named objects must be created, modified, and then moved from.

```
std::vector<std::string> readStrings(int numStrings) {
    std::vector<std::string> result;
```

```
    std::string string;
    while (numStrings--) {
        std::cin >> string;
        result.push_back(std::move(string));
    }
    return result;
}
```

In the above code, moving from both `string` and `result` is appropriate, as neither object needs to hold its value any longer after those values have respectively been transferred to an element of `result` and to the returned object. Both `string` and `result` are named objects. However, in only one case should `std::move` be applied to convert its operand from an lvalue to an xvalue and force a move to occur.

In all versions of C++, `return result;` permits (but does not require) the compiler to make the local variable `result` an alias for the returned object, obviating the need for any move at all. This optimization is known as *named-return-value optimization* (NRVO). In all versions since C++11, if the compiler declines to perform NRVO, then the *id-expression* `result` will be treated as an xvalue in the `return` statement, even though it is an lvalue when appearing elsewhere.

Writing `return std::move(result);` is not only unnecessary, but also detrimental to performance: NRVO does not occur when the operand of the `return` statement is anything other than the name of a local variable. Therefore, adding a call to `std::move` can force the compiler to actually create a separate `result` object on the stack frame of `readStrings` and then perform the move from that object to the returned object (often allocated on the stack frame of the caller).

Those learning C++ are taught to write `std::move(string)` because `string` is an lvalue referring to an object that no longer needs to hold its value after that value has been passed to the function `push_back`. I'll call this the *When to Move Rule*.

> `std::move` should be applied to an lvalue if the object to which the lvalue refers no longer needs to hold its value after the current operation.

While the When to Move Rule indicates some uses of `std::move` that are unnecessary (such as when the operand is of a trivially copyable type), in only a single case is following the When to Move Rule likely to be a pessimization, and I will refer to this case as the *When Not to Move Rule*:

> Notwithstanding the When to Move Rule, `std::move` should not be used when the operand of a `return` statement is the name of a local variable whose type is the same as the return type of the function (modulo top-level cv-qualification) since the use of `std::move` can inhibit NRVO.

The When Not to Move Rule is hard to learn and remember. Novices should not have to think about compiler optimizations that may or may not happen, and the scope of the exception seems arbitrary unless the developer understands the boundaries of NRVO. Because even experienced C++ programmers sometimes wonder why NRVO requires an exact match in types, one cannot expect a novice to remember the exact circumstances under which NRVO might happen (which contraindicates the use of `std::move`).

Thanks to the adoption of [P1155R3] into C++20 as a defect report (DR), the When Not to Move Rule can be simplified: "`std::move` should not be used when the operand of a `return` statement is the name of a local variable, full stop." This version of the rule is simpler than the version above. Even still, the fact that the When Not to Move Rule needs to be explained at all is a teachability burden. Novices should neither have to attempt to understand NRVO, nor should they have to think about lvalues automatically becoming rvalues in certain positions, especially considering that understanding which expressions are even lvalues is hard enough. Yet, these concepts are both required to explain why violating the When Not to Move Rule can make the performance of code only worse, never better.

In C++23, the When Not to Move Rule is actually part of the When to Move Rule, because [P2266R3] reclassified the name of a local variable as an xvalue when that name is the operand of a `return` statement. For the reasons explained in the previous paragraph, it is unclear whether shifting the complexity of the When Not to Move Rule into the definitions of the value categories makes the actual recommended practice any easier to understand.

§ 3 The solution

The conditions under which NRVO can occur for a `return` statement are specified in § [class.copy.elision]p1.1[1]: The operand must be an *id-expression* naming a nonvolatile object with automatic storage duration that is not a function parameter nor the parameter of a `catch` clause and whose type, ignoring cv-qualification, is the same as the return type of the function. Call these expressions *NRVO eligible*. I propose that when the expression `E` is NRVO eligible, the following expressions should also be NRVO eligible:

- `std::move(E)` or, more generally, `F(E)` when `F` is an *id-expression* that denotes `std::move` after name lookup
- `static_cast<T>(E)`, where `T` is any *type-id* denoting an rvalue reference to the return type
- `(T)E`, where `T` is any *type-id* denoting an rvalue reference to the return type
- `T(E)`, where `T` is any *simple-type-specifier* denoting an rvalue reference to the return type

Redundant parentheses are allowed around both *E* and the cast expression, under § [expr.prim.paren].

The first of the above forms is the one beginners will use most often, but the remaining forms should also be allowed for consistency.

I further propose that the above change should be considered a DR so that compilers will implement it in earlier language versions.

After this change is made to the Standard, the When Not to Move Rule will no longer need to be taught.

Note, however, that adopting this proposal will neither force implementations to perform NRVO in any situation where they currently do not, nor will any implementations be forced to generate code for `return std::move(x);` that is equally optimized as the code generated for `return x;`. This proposal would only *allow* implementations to do so.

§ 4 Should the core language have special cases for library entities?

The change proposed in this paper would give special core-language treatment for the Standard Library entity `std::move`. This change is unlikely to present any implementation challenges, since Clang and GCC have already started treating expressions of the form `std::move(E)` as if the corresponding cast had been written explicitly (thus avoiding the overhead of instantiating the function template `std::move`). Still, one might object that such special treatment is unusual when it occurs in the core language specification.

However, such special cases already exist within the core language. This is illustrated by the following nonexhaustive list of such special cases:

— `std::byte` is exempt from strict aliasing: A glvalue of type `std::byte` can be used to access the value of an object of any type. No other enumeration type has this permission.

— `std::initializer_list` has special rules for deduction: When a function template has a parameter of type `std::initializer_list<T>`, with `T` being a type template argument, `T` can be deduced when the corresponding argument is a *braced-init-list*. No other class template supports such deduction.

— Member functions of `std::allocator<T>` have special permission to be called during constant evaluation despite being defined in terms of expressions that are not permitted in constant evaluation.

— Structured bindings use `std::tuple_size` and `std::tuple_element`.

The existence of these special cases reflects the design principle that the core language is not truly separate from the Standard Library; they are simply described in different sections of the Standard.

§ 5 How many programs will be broken by this change?

The change proposed by this paper can alter the behavior of existing code, because whether NRVO is applied affects the observable behavior of a program. Such a change can be considered *breaking* for a program whose correct functioning depends on NRVO *not* occurring, e.g., a test driver that counts the number of times a move constructor is called. A programmer who is aware of NRVO would consider writing such code inadvisable, but others might do so anyway, unaware of the impact.

I expect that the actual amount of breakage caused by improvements to NRVO is relatively small; were it not so, then discretionary improvements to NRVO under the current rules (i.e., a newer version of a compiler choosing to implement NRVO in a situation where an older version did not) would also frequently cause similar problems. However, this issue is rarely mentioned among the issues that need to be fixed in a codebase during a toolchain upgrade.

Some users might be deliberately using `std::move` to suppress NRVO under circumstances where NRVO occurring is undesirable. For those users, the change proposed by this paper would be breaking, even if other changes to NRVO might not be breaking. This pattern probably occurs infrequently, because deliberately suppressing NRVO is rarely desirable.

If this proposal receives positive feedback in the EWG, the amount of breakage could be gauged more precisely by running test suites for large open source projects after implementing the proposed change in a compiler, should EWG consider such implementation experience necessary.

§ 6 Alternative solutions

As an alternative, an expression could be considered NRVO-eligible when it is an xvalue that refers to a local variable whose type is the same as the function return type (modulo top-level cv-qualification). This rule would make NRVO eligibility a runtime property in theory; in practice, implementations would be able to perform NRVO only when they can prove that the operand of the `return` statement always refers to one particular local variable.

Compared with the proposed solution, this alternative solution has the elegance of avoiding any special case for `std::move`. However, by allowing NRVO in cases that lack clarity regarding whether the compiler will be able to prove that NRVO is allowed, this alternative rule can be a source of nonportable behavior. The practical downside seems to outweigh the theoretical elegance of not making `std::move` a special case. An additional downside is that, under this alternative solution, how to write a `return` statement that forces NRVO *not* to occur, which might be desirable in some circumstances, is unclear.

Another alternative solution that would reduce implementation divergence, while also avoiding special-casing of `std::move`, is to require a trial constant evaluation when the operand of a `return` statement is an xvalue with the same type as the function return type (modulo top-level cv-qualification); if this trial constant evaluation succeeds and the result refers to a local variable, then NRVO would be permitted to occur. If the result of evaluating the operand depends on the value of any function parameters or other information that is known only at run time, then the trial constant evaluation will fail.

Such a trial constant evaluation would perhaps be detrimental to compile times, might pose implementation challenges, might be difficult to specify and give rise to CWG issues, and might fail to achieve the objective of avoiding implementation divergence due to its complexity. (For example, what should the implementation do if the result of the trial constant evaluation differs from what would actually occur at run time, e.g., because of the use of `if consteval`?) Finally, backporting such a trial constant evaluation to earlier language versions is likely to be unfeasible (i.e., adopting this alternative as a DR), which means that pessimization would continue to be required in older language modes.

§ 7 Wording

The proposed wording is relative to [N4958].

Edit § [class.copy.elision]p1.1 as shown.

> in a `return` statement in a function with a class return type, when the *expression* is NRVO eligible (see below) and refers to~~the name of~~ a non-volatile object with automatic storage duration (other than a function parameter or a variable introduced by the *exception-declaration* of a *handler* ([except.handle])) with the same type (ignoring cv-qualification) as the function return type, the copy/move operation can be omitted by constructing the object directly into the function call's return object

Edit § [class.copy.elision]p1.2 as shown.

> in a *throw-expression* ([expr.throw]), when the operand is NRVO eligible (see below) and refers to~~the name of~~ a non-volatile object with automatic storage duration (other than a function or catch-clause parameter) that belongs to a scope that does not contain the innermost enclosing *compound-statement* associated with a *try-block* (if there is one), the copy/move operation can be omitted by constructing the object directly into the exception object

Add a new paragraph after § [class.copy.elision]p1:

> An expression is *NRVO eligible* if it is:
>
> — an *id-expression*,
>
> — `(` *E* `)`, where *E* is NRVO eligible, or

— one of the below expressions, where *E* is NRVO eligible and `T` is the rvalue reference type whose referenced type is that of *E*:

— *F*`(`*E*`)`, where *F* is a (possibly parenthesized) *id-expression* denoting `::std::move` ([forward])

— `static_cast<`*type-id*`>(`*E*`)` or `(`*type-id*`)`*E*, where the *type-id* denotes `T`

— *simple-type-specifier*`(`*E*`)`, where the *simple-type-specifier* denotes `T`

§ 8 References

[N4958] Thomas Köppe. 2023-08-14. Working Draft, Programming Languages — C++. https://wg21.link/n4958

[P1155R3] Arthur O'Dwyer, David Stone. 2019-06-17. More implicit moves. https://wg21.link/p1155r3

[P2266R3] Arthur O'Dwyer. 2022-03-26. Simpler implicit move. https://wg21.link/p2266r3

---

1. All citations to the Standard are to working draft [N4958] unless otherwise specified.↩